

Introducción al aprendizaje profundo

Diego Milone

Inteligencia Computacional
Departamento de Informática
FICH - Universidad Nacional del Litoral

Organización

Introducción

Aprendizaje automático: clasificaciones...

Aprendizaje automático en el tiempo...

Programación diferenciable

Muchos datos, muchas neuronas

Qué es y qué no es?

Modos directo y reverso

Aprendizaje con ∂P

Entrenando neuronas

Entrenando con programas diferenciables

Métodos clásicos de aprendizaje automático

- Supervisados
- No supervisados
- Aprendizaje por refuerzo

Métodos clásicos de aprendizaje automático

- Supervisados
 - Clasificación
 - Naive Bayes
 - Máquinas de vectores de soporte (SVM)
 - Árboles de decisión
 - k-vecinos más cercanos (k-NN)
 - Regresión
 - Regresión lineal
 - Regresión logística
 - No supervisados
-
- Aprendizaje por refuerzo

Métodos clásicos de aprendizaje automático

- Supervisados
 - Clasificación
 - Naive Bayes
 - Máquinas de vectores de soporte (SVM)
 - Árboles de decisión
 - k-vecinos más cercanos (k-NN)
 - Regresión
 - Regresión lineal
 - Regresión logística
- No supervisados
 - Clustering
 - k-medias
 - DBSCAN
 - Reducción dimensional
 - Análisis de componentes principales (PCA)
 - Factorización de matrices no negativas
- Aprendizaje por refuerzo

Procesos de aprendizaje

- Tipos de aprendizaje
- Reglas de aprendizaje

Procesos de aprendizaje

- Tipos de aprendizaje
 - Supervisado
 - No supervisado
 - Por refuerzo
 - Híbridos
- Reglas de aprendizaje

Procesos de aprendizaje

- Tipos de aprendizaje
 - Supervisado
 - No supervisado
 - Por refuerzo
 - Híbridos
- Reglas de aprendizaje
 - Corrección de error
 - Competitivo
 - Hebbiano
 - Boltzman

Procesos de aprendizaje

- Tipos de aprendizaje
 - Supervisado
 - No supervisado
 - Por refuerzo
 - Híbridos
- Reglas de aprendizaje
 - Corrección de error
 - Competitivo
 - Hebbiano
 - Boltzman
- Y muchos más...

semi-supervised, self-supervised, inductive, deductive, transductive, incremental, adaptive, online, active, multi-label, multi-task, multi-instance, zero-shot, one-shot, few-shot, one-class, generative adversarial, federated, meta-learning, feature learning, domain adaptation, knowledge distillation, transfer learning,...

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
- Redes recurrentes (recurrent/with feedback)
- Otros modelos híbridos

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
 - Perceptrones simples y de una capa
 - Perceptrón multicapa
 - Redes con funciones de base radial
 - ...
- Redes recurrentes (recurrent/with feedback)
- Otros modelos híbridos

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
 - Perceptrones simples y de una capa
 - Perceptrón multicapa
 - Redes con funciones de base radial
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Redes de Kohonen
 - Redes de Hopfield
 - Redes de Boltzmann
 - Redes de Elman/Jordan
 - Modelos de resonancia adaptativa
 - ...
- Otros modelos híbridos

Arquitecturas neuronales tradicionales

- Redes de propagación hacia adelante (feedforward)
 - Perceptrones simples y de una capa
 - Perceptrón multicapa
 - Redes con funciones de base radial
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Redes de Kohonen
 - Redes de Hopfield
 - Redes de Boltzmann
 - Redes de Elman/Jordan
 - Modelos de resonancia adaptativa
 - ...
- Otros modelos híbridos
 - Parcialmente recurrentes
 - Parcialmente conectados
 - Redes modulares
 - ...

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
- Redes recurrentes (recurrent/with feedback)
- Otros modelos y métodos de entrenamiento

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
 - Deep Belief Network (DBN)
 - Convolutional Neural Network (CNN)
 - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
 - ...
- Redes recurrentes (recurrent/with feedback)
- Otros modelos y métodos de entrenamiento

Arquitecturas profundas

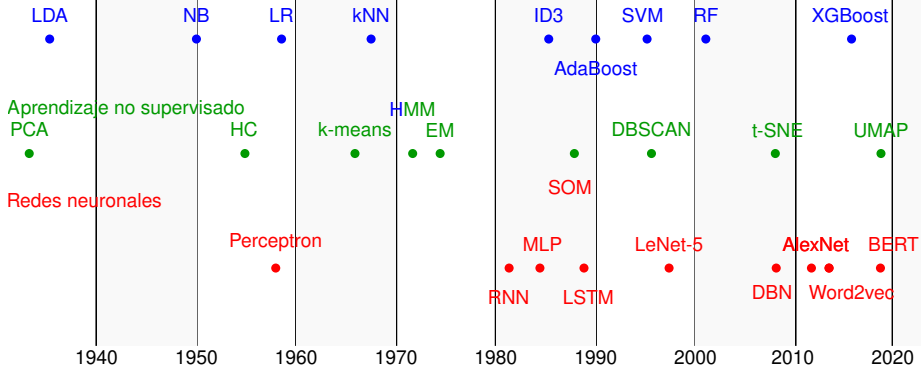
- Redes de propagación hacia adelante (feedforward)
 - Deep Belief Network (DBN)
 - Convolutional Neural Network (CNN)
 - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Gated Recurrent Unit (GRU)
 - Long Short-Term Memory (LSTM)
 - Attention Mechanisms, Transformers
 - ...
- Otros modelos y métodos de entrenamiento

Arquitecturas profundas

- Redes de propagación hacia adelante (feedforward)
 - Deep Belief Network (DBN)
 - Convolutional Neural Network (CNN)
 - ... LeNet, AlexNet, ZF Net, GoogleLeNet, VGGNet, U-Net, ResNet, DenseNet, CapsuleNet, GCN...
 - ...
- Redes recurrentes (recurrent/with feedback)
 - Gated Recurrent Unit (GRU)
 - Long Short-Term Memory (LSTM)
 - ~~Attention Mechanisms, Transformers~~
 - ...
- Otros modelos y métodos de entrenamiento
 - Generative Adversarial Networks (GAN)
 - Deep Reinforcement Learning (DRL)
 - Bayesian Deep Learning (BDL)
 - Normalizing Flows
 - Diffusion Models
 - ...

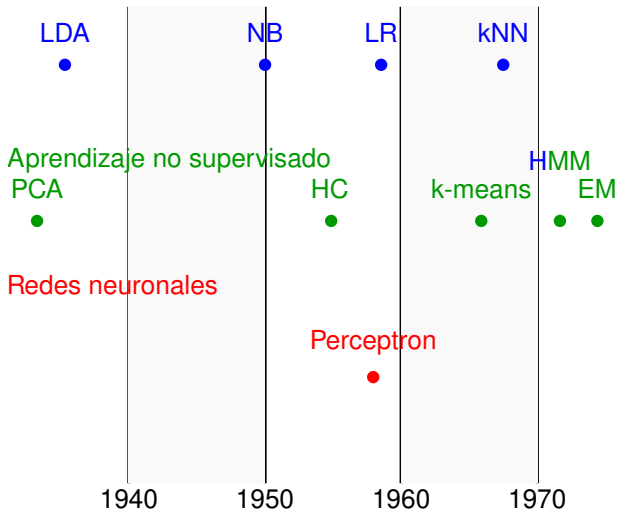
Evolución histórica

Aprendizaje supervisado

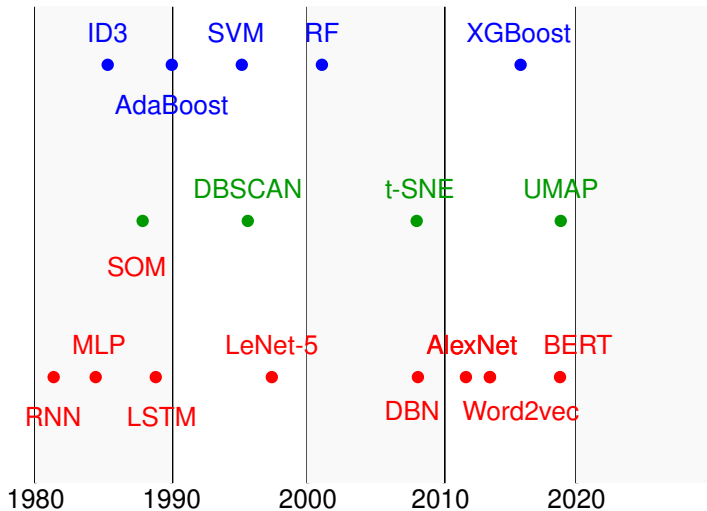


Evolución histórica

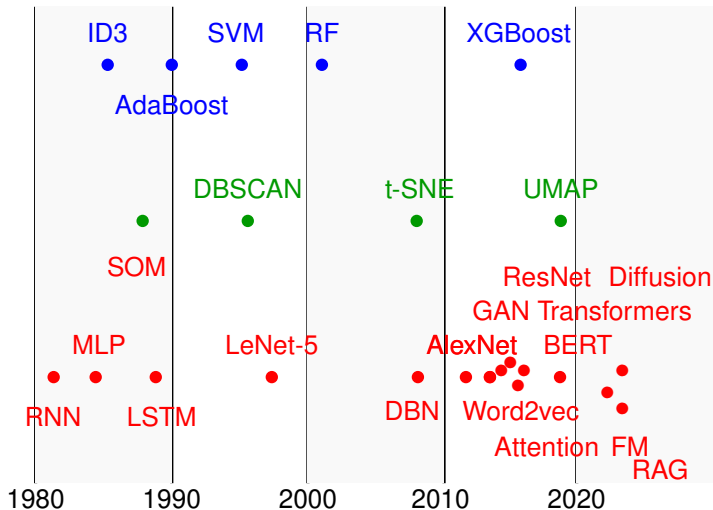
Aprendizaje supervisado



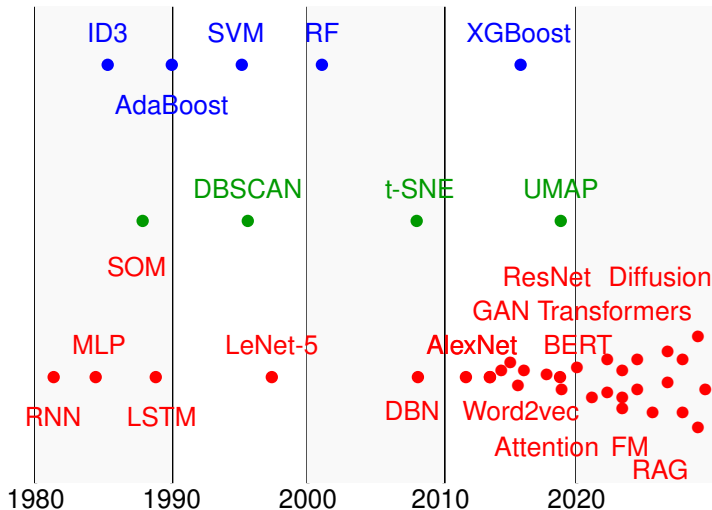
Evolución histórica

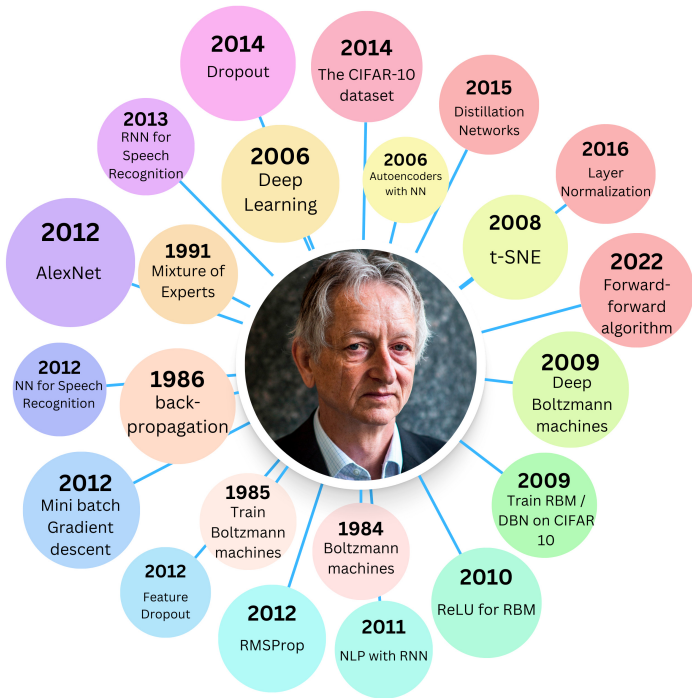


Evolución histórica

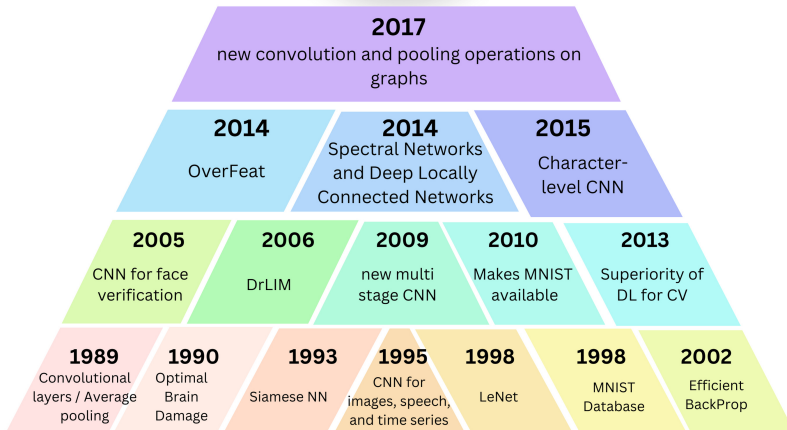


Evolución histórica

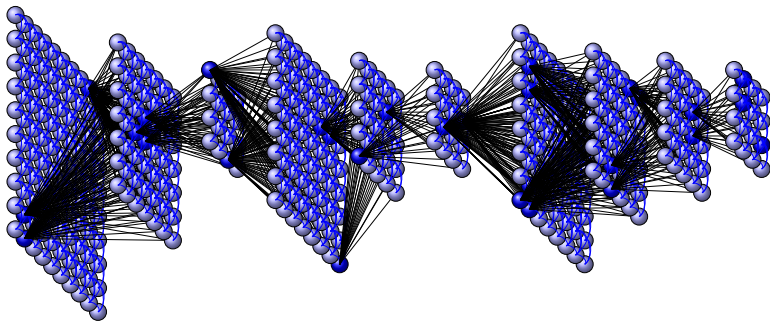








¿Qué pasa si conectamos muuuchas neuronas?



Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**
- Desvanecimiento del gradiente

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**
- Desvanecimiento del gradiente ←
 - Pre-entrenamiento auto-supervisado,
 - Normalización por lotes,
 - Otros condimentos: **ReLU**, dropout, pooling...

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**
- Desvanecimiento del gradiente ←
 - Pre-entrenamiento auto-supervisado,
 - Normalización por lotes,
 - Otros condimentos: **ReLU**, dropout, pooling...
- Dificultad para obtener los gradientes en modelos complejos y programarlos a gran escala

Qué pasa si conectamos muuuchas neuronas?

- Necesidad de contar con grandes cantidades de datos para entrenar
 - ← Revolución de los **grandes datos**
- Alto costo computacional
 - ← Unidades de procesamiento gráfico **GPUs**
- Desvanecimiento del gradiente ←
 - Pre-entrenamiento auto-supervisado,
 - Normalización por lotes,
 - Otros condimentos: **ReLU**, dropout, pooling...
- Dificultad para obtener los gradientes en modelos complejos y programarlos a gran escala
 - ← **Programación diferenciable** ∂P

Programación diferenciable

LeCun Manifesto

**Yann LeCun** ✓

January 5, 2018 · 🌐

OK, Deep Learning has outlived its usefulness as a buzz-phrase.
Deep Learning est mort. Vive Differentiable Programming!

Yeah, Differentiable Programming is little more than a rebranding of the modern collection of Deep Learning techniques, the same way Deep Learning was a rebranding of the modern incarnations of neural nets with more than two layers.

But the important point is that people are now building a new kind of software by assembling networks of parameterized functional blocks and by training them from examples using some form of gradient-based optimization.

An increasingly large number of people are defining the networks procedurally in a data-dependent way (with loops and conditionals), allowing them to change dynamically as a function of the input data fed to them. It's really very much like a regular program, except it's parameterized, automatically differentiated, and trainable/optimizable. Dynamic networks have become increasingly popular (particularly for NLP), thanks to deep learning frameworks that can handle them such as PyTorch and Chainer (note: our old deep learning framework Torch could handle a particular kind of dynamic nets called Graph Transformer Networks, back

Programación diferenciable

También conocida como:

- ∂P (differentiable programming)
- Automatic Differentiation
- Algorithmic Differentiation
- Dynamic Computational Graphs
- *autodiff*
- *autograd*
- ~~Symbolic Differentiation~~
- ~~Numerical Differentiation~~

Programación diferenciable: mapa logístico

Modelo poblacional

$$\frac{d\mathcal{L}}{dt} = r\mathcal{L} \left(1 - \frac{\mathcal{L}}{K}\right),$$

donde:

- $\mathcal{L}$ es la cantidad de individuos,
- r es la tasa de crecimiento y
- el factor $(1 - \mathcal{L}/K)$ limita el crecimiento.

Programación diferenciable: mapa logístico

Modelo poblacional

$$\frac{d\mathcal{L}}{dt} = r\mathcal{L} \left(1 - \frac{\mathcal{L}}{K}\right),$$

donde:

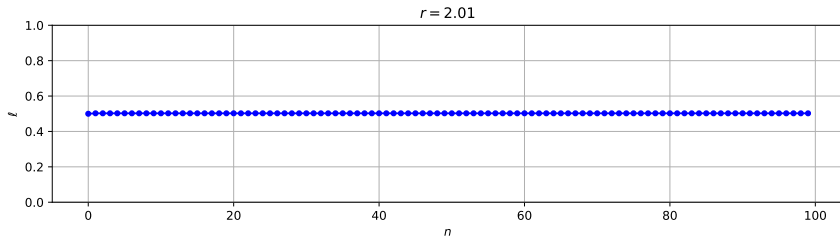
- $\mathcal{L}$ es la cantidad de individuos,
- r es la tasa de crecimiento y
- el factor $(1 - \mathcal{L}/K)$ limita el crecimiento.

La versión discreta puede simplificarse como

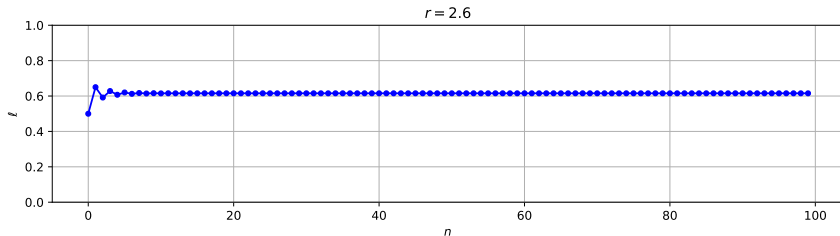
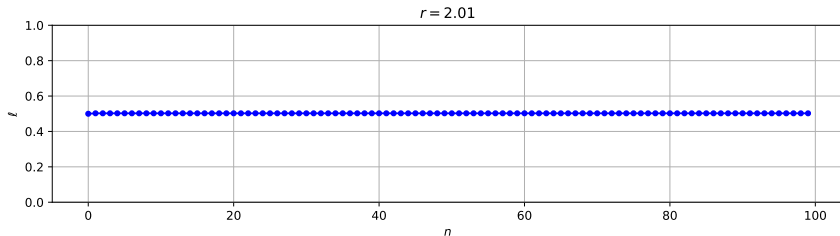
$$\ell_{n+1} = r\ell_n(1 - \ell_n),$$

donde ahora ℓ_n es la población normalizada.

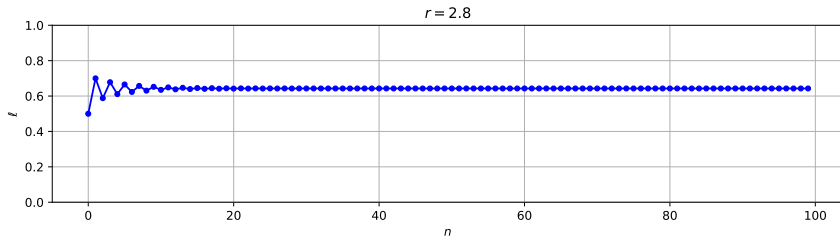
Mapa logístico



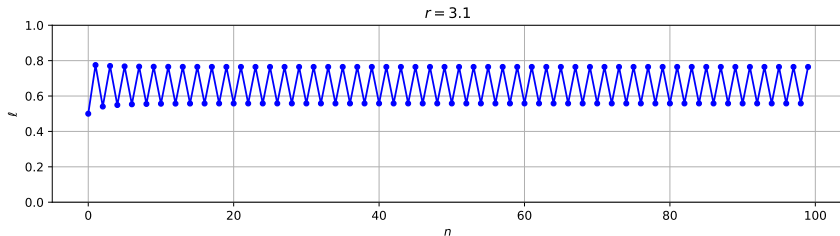
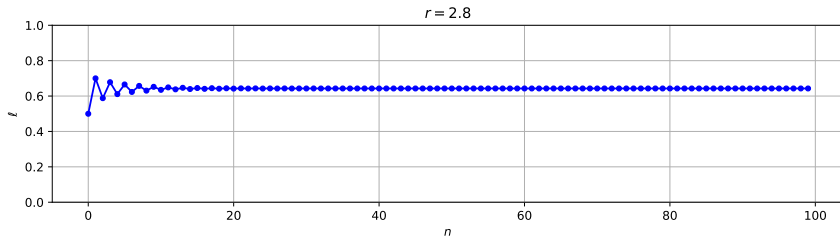
Mapa logístico



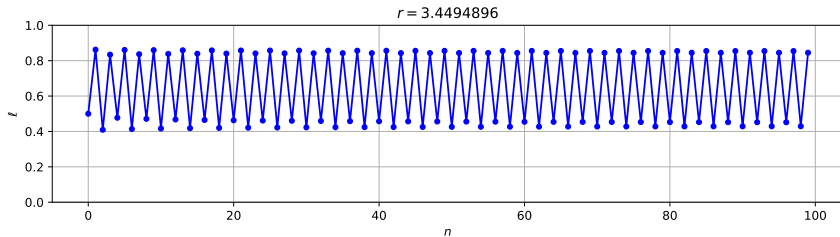
Mapa logístico



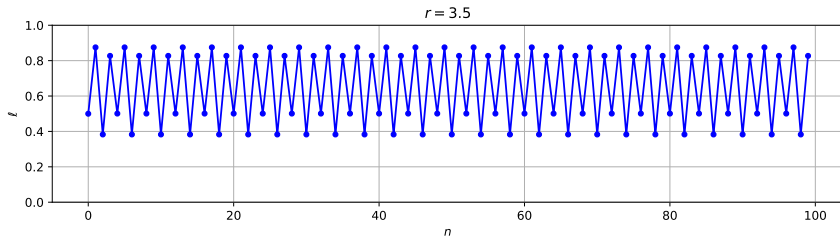
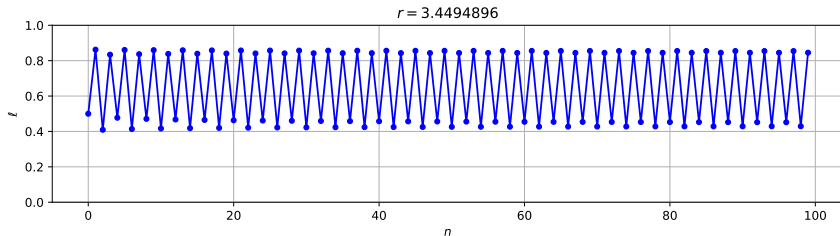
Mapa logístico



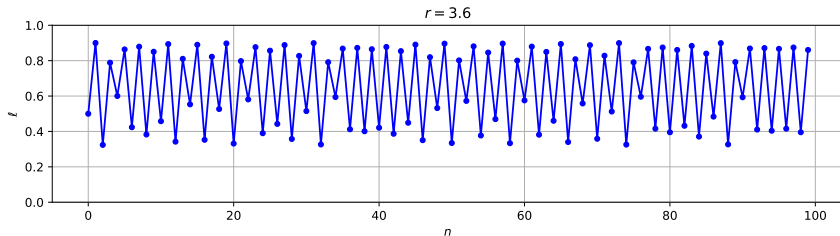
Mapa logístico



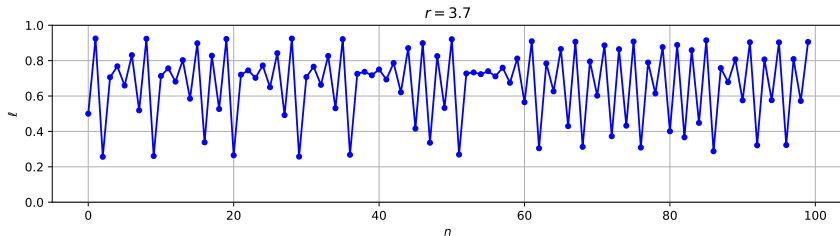
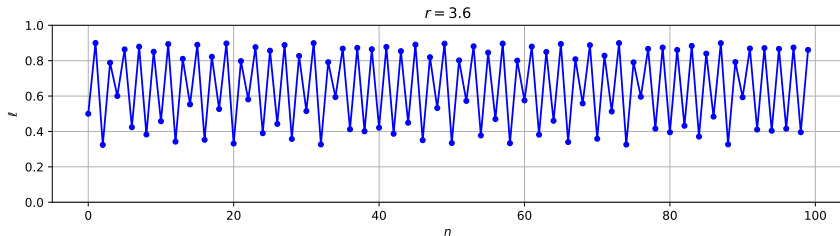
Mapa logístico



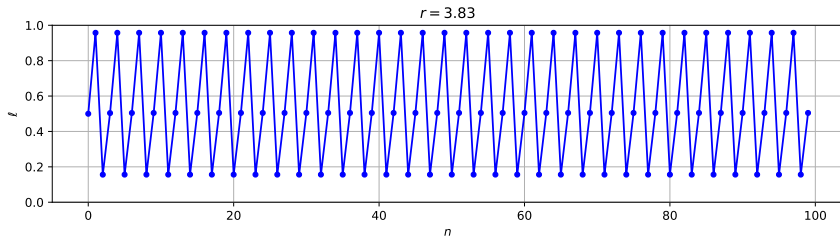
Mapa logístico



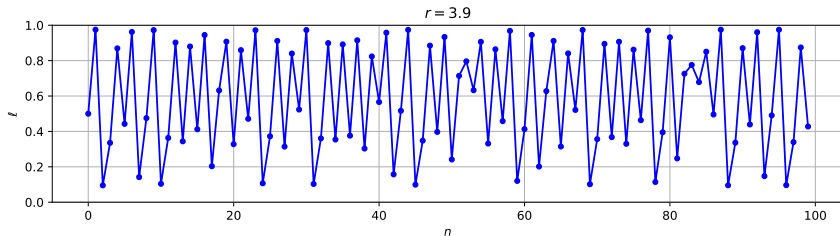
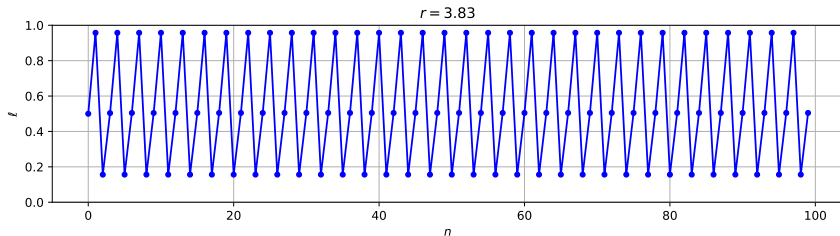
Mapa logístico



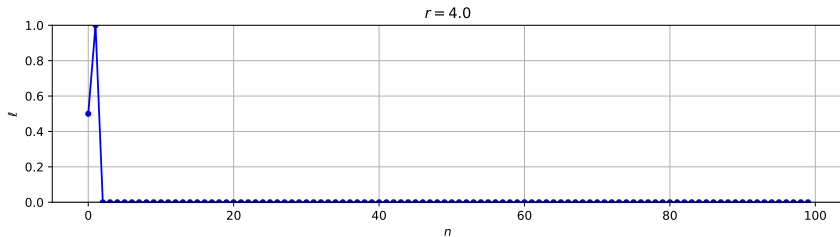
Mapa logístico



Mapa logístico



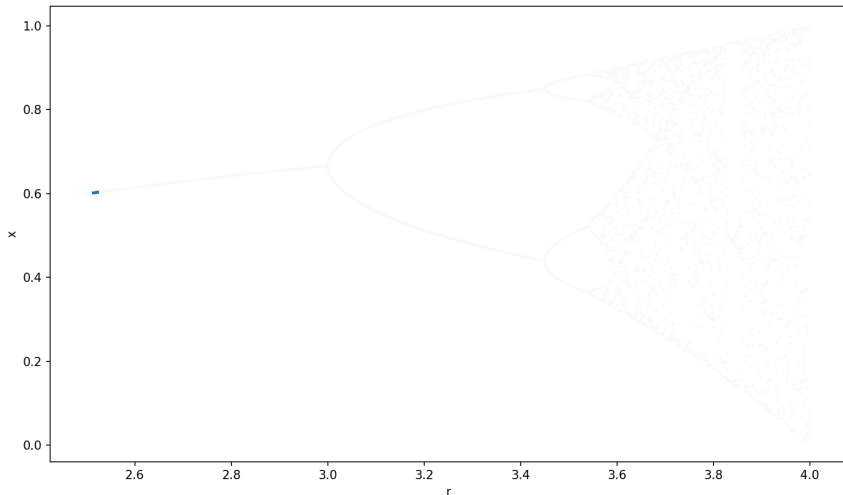
Mapa logístico



Mapa logístico

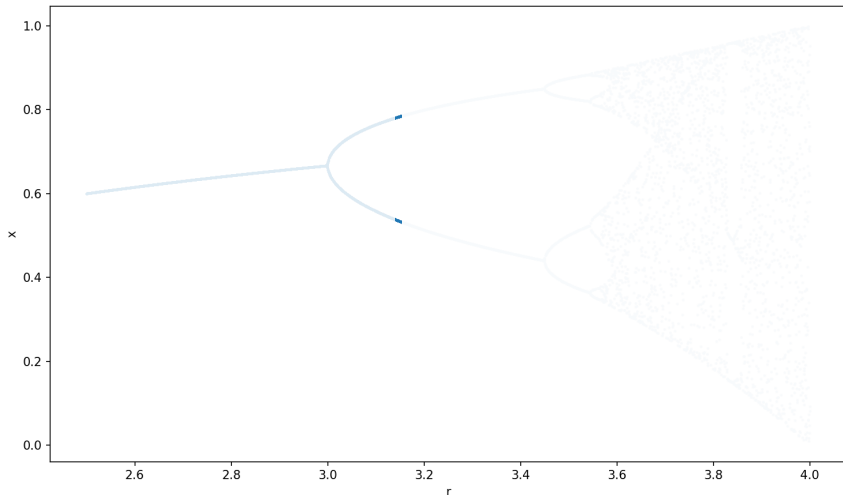
Programación diferenciable: mapa logístico

Bifurcaciones y caos en el mapa logístico

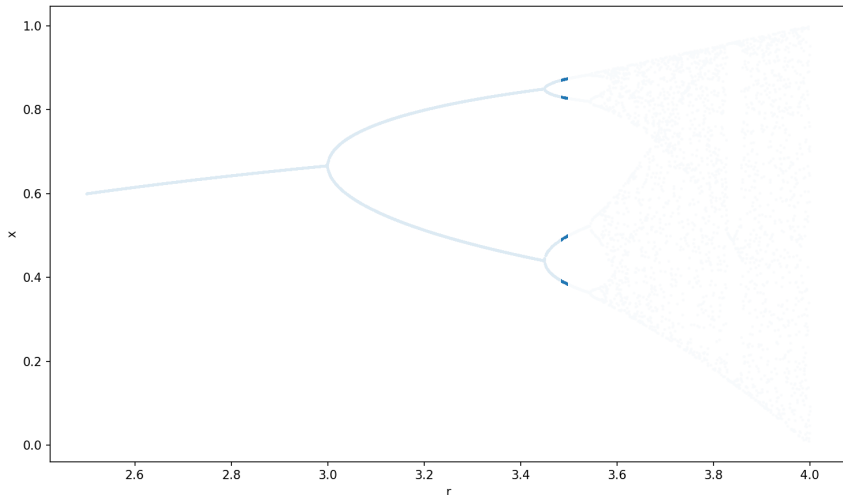


Programación diferenciable: mapa logístico

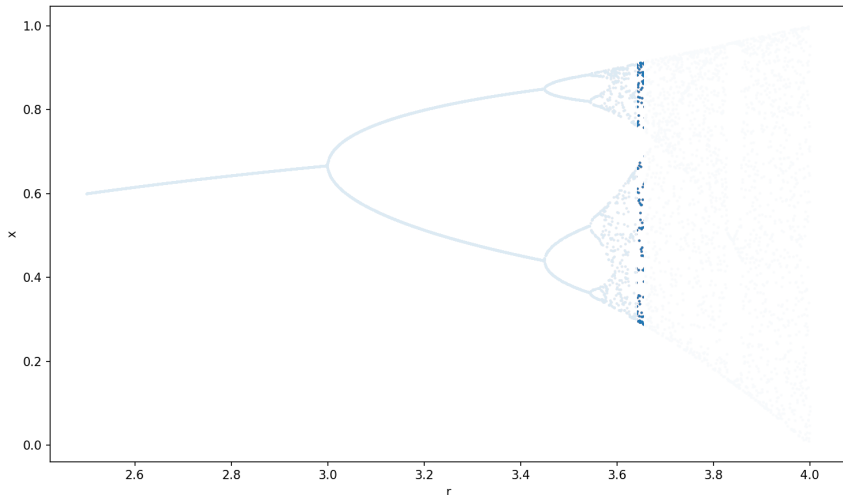
Bifurcaciones y caos en el mapa logístico



Bifurcaciones y caos en el mapa logístico

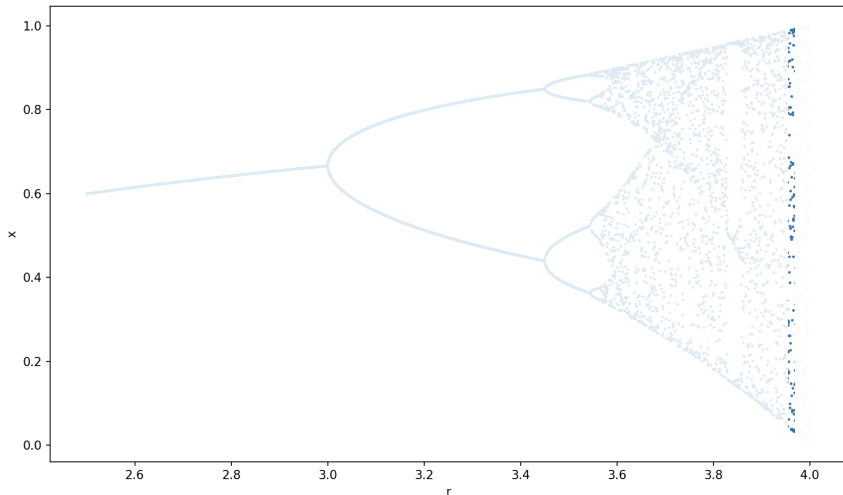


Bifurcaciones y caos en el mapa logístico



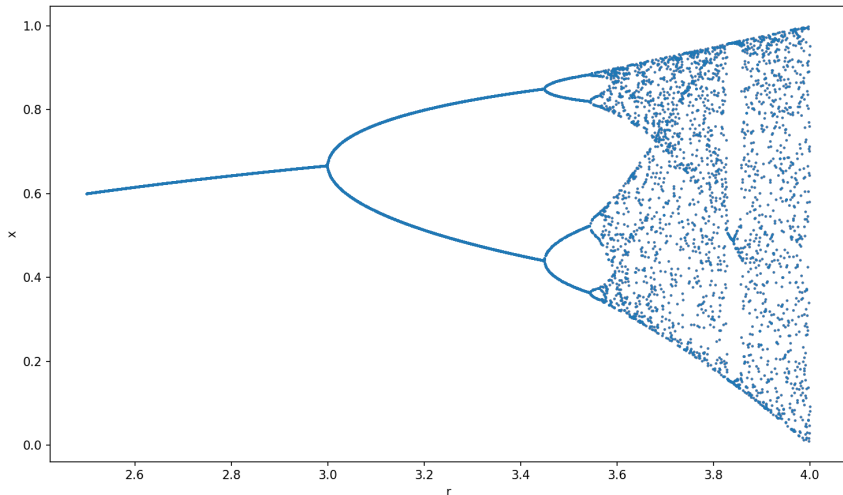
Programación diferenciable: mapa logístico

Bifurcaciones y caos en el mapa logístico



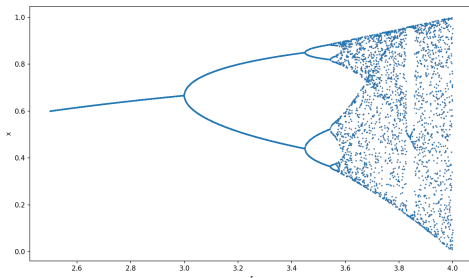
Programación diferenciable: mapa logístico

Bifurcaciones y caos en el mapa logístico



Programación diferenciable: mapa logístico

Bifurcaciones y caos en el mapa logístico



Supongamos que tenemos que calcular sus derivadas¹

$$\ell_{n+1} = r\ell_n(1 - \ell_n), \quad \ell_1 = x$$

con respecto a x , a lo largo de las iteraciones n .

¹ A.G. Baydin et al. Automatic differentiation in machine learning: A survey. JMLR, 2018.

Derivando el mapa logístico

Con $r = 4$, por ejemplo, tenemos: $\ell_{n+1} = 4\ell_n(1 - \ell_n)$, $\ell_1 = x$

Derivando el mapa logístico

Con $r = 4$, por ejemplo, tenemos: $\ell_{n+1} = 4\ell_n(1 - \ell_n)$, $\ell_1 = x$

n ℓ_n

1 x

2 $4x(1 - x)$

3 $16x(1-x)(1-2x)^2$

4 $64x(1-x)(1-2x)^2$
 $(1 - 8x + 8x^2)^2$

Derivando el mapa logístico

Con $r = 4$, por ejemplo, tenemos: $\ell_{n+1} = 4\ell_n(1 - \ell_n)$, $\ell_1 = x$

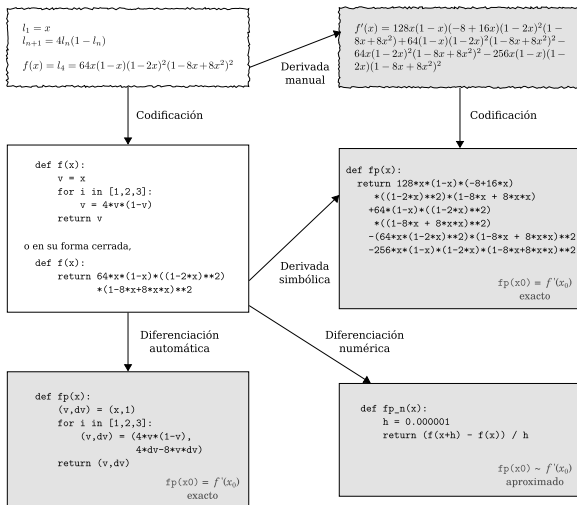
n	ℓ_n	$\frac{d}{dx}\ell_n$
1	x	1
2	$4x(1-x)$	$4(1-x) - 4x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2 - 16x(1-2x)^2 - 64x(1-x)(1-2x)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$

Derivando el mapa logístico

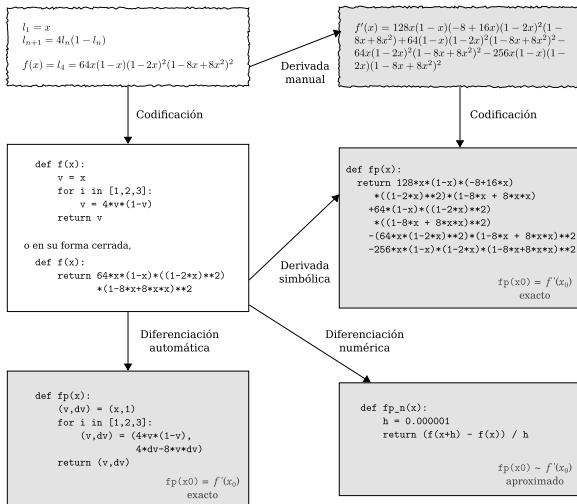
Con $r = 4$, por ejemplo, tenemos: $\ell_{n+1} = 4\ell_n(1 - \ell_n)$, $\ell_1 = x$

n	ℓ_n	$\frac{d}{dx}\ell_n$	y simplificando...
1	x	1	1
2	$4x(1-x)$	$4(1-x) - 4x$	$4 - 8x$
3	$16x(1-x)(1-2x)^2$	$16(1-x)(1-2x)^2 - 16x(1-2x)^2 - 64x(1-x)(1-2x)$	$16(1 - 10x + 24x^2 - 16x^3)$
4	$64x(1-x)(1-2x)^2(1-8x+8x^2)^2$	$128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$	$64(1 - 42x + 504x^2 - 2640x^3 + 7040x^4 - 9984x^5 + 7168x^6 - 2048x^7)$

Diversas formas de obtener la derivada



Tomemos el mapa logístico en $n = 4$



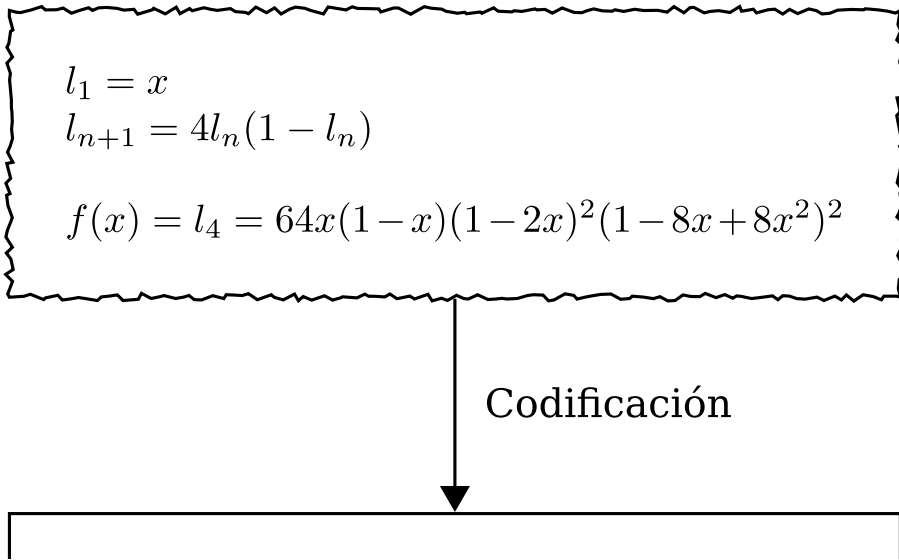
Tomemos el mapa logístico en $n = 4$

$$l_1 = x$$

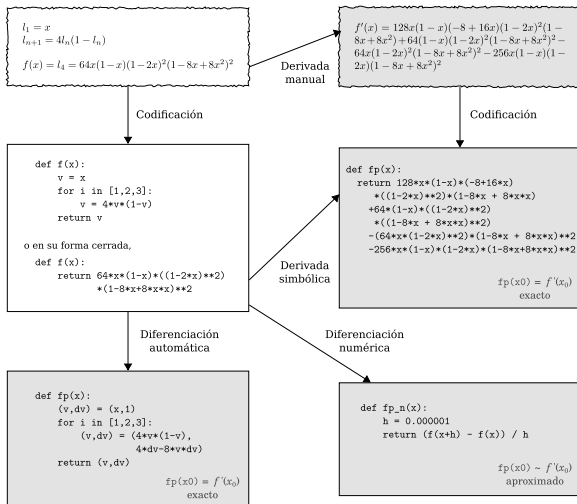
$$l_{n+1} = 4l_n(1 - l_n)$$

$$f(x) = l_4 = 64x(1-x)(1-2x)^2(1-8x+8x^2)^2$$

Codificación



Programando sería algo así como...



Programando sería algo así como...

Codificación

```
def f(x):
    v = x
    for i in [1,2,3]:
        v = 4*v*(1-v)
    return v
```

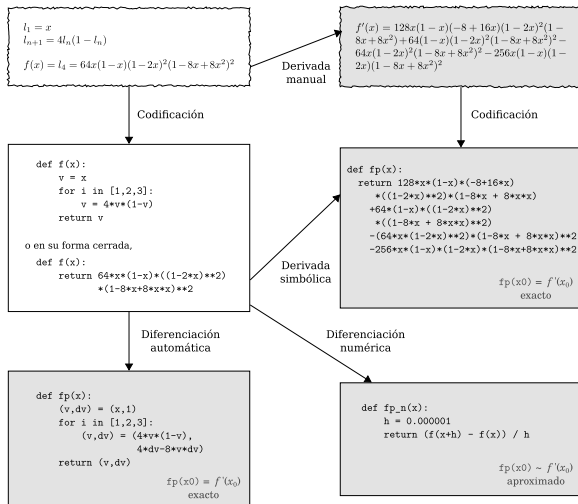
o en su forma cerrada,

```
def f(x):
    return 64*x*(1-x)*((1-2*x)**2)
           *(1-8*x+8*x*x)**2
```

Derivada
simbólica

```
def f(x):
    return
    +
    +
    -
    -2
```

La derivada que obtuvimos manualmente era:



La derivada que obtuvimos manualmente era:

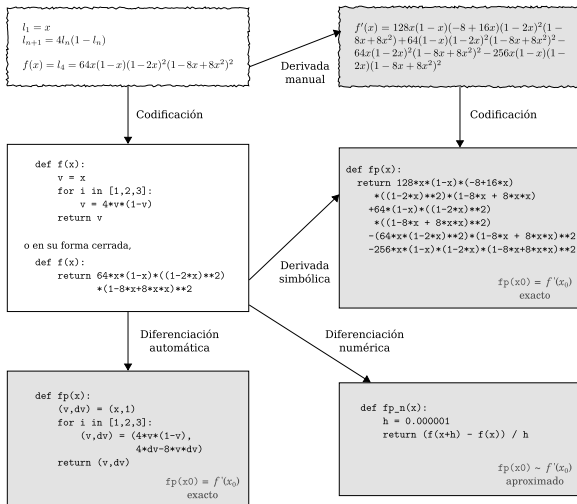
Derivada
manual

$$f'(x) = 128x(1-x)(-8+16x)(1-2x)^2(1-8x+8x^2) + 64(1-x)(1-2x)^2(1-8x+8x^2)^2 - 64x(1-2x)^2(1-8x+8x^2)^2 - 256x(1-x)(1-2x)(1-8x+8x^2)^2$$

Codificación

```
def fp(x):  
    return 128*x*(1-x)*(-8+16*x)
```

Con derivación simbólica o programando la manual



Con derivación simbólica o programando la manual

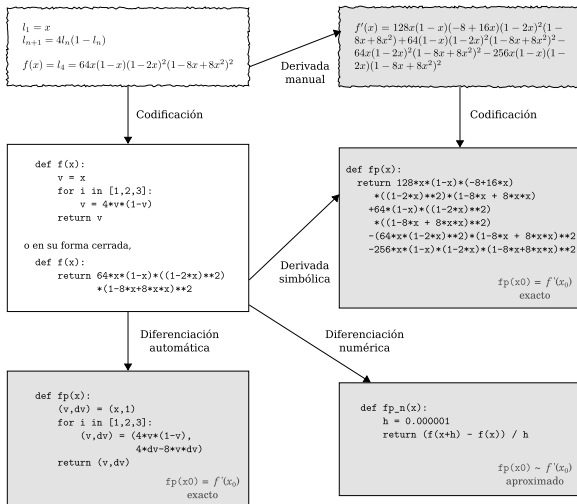
Codificación

```
def fp(x):
    return 128*x*(1-x)*(-8+16*x)
        *((1-2*x)**2)*(1-8*x + 8*x*x)
    +64*(1-x)*((1-2*x)**2)
        *((1-8*x + 8*x*x)**2)
    -(64*x*(1-2*x)**2)*(1-8*x + 8*x*x)**2
    -256*x*(1-x)*(1-2*x)*(1-8*x+8*x*x)**2
```

Derivada
simbólica

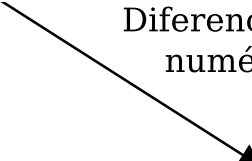
$fp(x_0) = f'(x_0)$
exacto

Diferenciación numérica



Diferenciación numérica

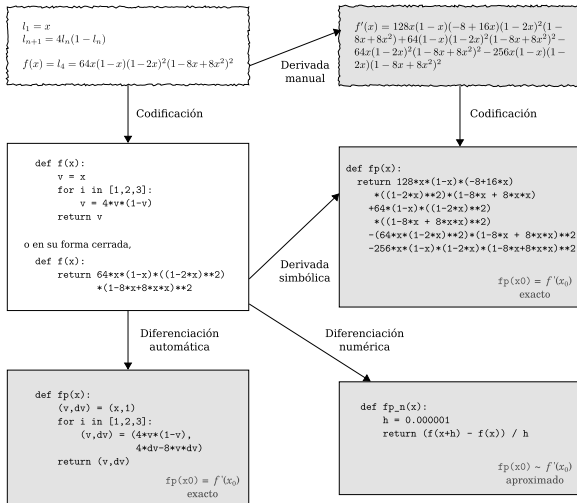
Diferenciación
numérica



```
def fp_n(x):  
    h = 0.000001  
    return (f(x+h) - f(x)) / h
```

$fp(x_0) \sim f'(x_0)$
aproximado

Derivación automática



Derivación automática

Diferenciación
automática

Di

```
def fp(x):  
    (v,dv) = (x,1)  
    for i in [1,2,3]:  
        (v,dv) = (4*v*(1-v),  
                  4*dv-8*v*dv)  
    return (v,dv)
```

$fp(x_0) = f'(x_0)$
exacto

Derivación automática: exacta y simple



```
def f(x):
    v = x
    for i in [1, 2, 3]:
        v = 4*v*(1-v)
    return v

def fp(x):
    (v, dv) = (x, 1)
    for i in [1, 2, 3]:
        (v, dv) = (4*v*(1-v), 4*dv-8*v*dv)
    return (v, dv)
```

Resumen: ventajas de la derivación automática

Resumen: ventajas de la derivación automática

- **Exactitud:** calcula derivadas exactas, sin la acumulación de errores por aproximación que posee la diferenciación numérica. Ayuda a evitar el desvanecimiento de los gradientes en arquitecturas profundas.

Resumen: ventajas de la derivación automática

- **Exactitud:** calcula derivadas exactas, sin la acumulación de errores por aproximación que posee la diferenciación numérica. Ayuda a evitar el desvanecimiento de los gradientes en arquitecturas profundas.
- **Evita la explosión de expresiones** (*expression swell*): no genera expresiones simbólicas largas y redundantes por falta de una adecuada simplificación. Propaga los valores numéricos de las derivadas, evitando arribar a ecuaciones crípticas y poco interpretables.

Resumen: ventajas de la derivación automática

- **Exactitud:** calcula derivadas exactas, sin la acumulación de errores por aproximación que posee la diferenciación numérica. Ayuda a evitar el desvanecimiento de los gradientes en arquitecturas profundas.
- **Evita la explosión de expresiones** (*expression swell*): no genera expresiones simbólicas largas y redundantes por falta de una adecuada simplificación. Propaga los valores numéricos de las derivadas, evitando arribar a ecuaciones crípticas y poco interpretables.
- **Automatización y eficiencia:** propaga las derivadas mediante la regla de la cadena a partir de operadores básicos, integrando su cálculo en la evaluación del mismo código. La evaluación de las derivadas se hace con una complejidad similar a la de la función original.

Resumen: ventajas de la derivación automática

- **Prototipado rápido:** facilita la reutilización de código y el desarrollo de nuevos modelos sin requerir derivación simbólica o manual para cada nueva idea (lo que hace muy lento el proceso).

Resumen: ventajas de la derivación automática

- **Prototipado rápido:** facilita la reutilización de código y el desarrollo de nuevos modelos sin requerir derivación simbólica o manual para cada nueva idea (lo que hace muy lento el proceso).
- **Flexibilidad:** permite diferenciar algoritmos con control de flujo (bucles, condicionales, recursión). No requiere que el modelo esté definido de forma cerrada, mediante una expresión explícita.

Resumen: ventajas de la derivación automática

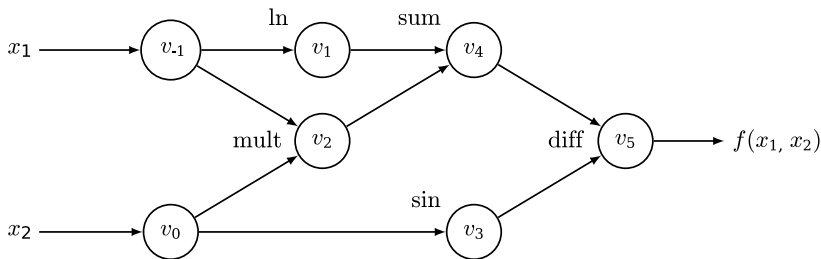
- **Prototipado rápido:** facilita la reutilización de código y el desarrollo de nuevos modelos sin requerir derivación simbólica o manual para cada nueva idea (lo que hace muy lento el proceso).
- **Flexibilidad:** permite diferenciar algoritmos con control de flujo (bucles, condicionales, recursión). No requiere que el modelo esté definido de forma cerrada, mediante una expresión explícita.
- **Escalabilidad:** es viable para modelos con billones de parámetros y millones de operaciones complejas.

Derivación automática: grafo de primitivas

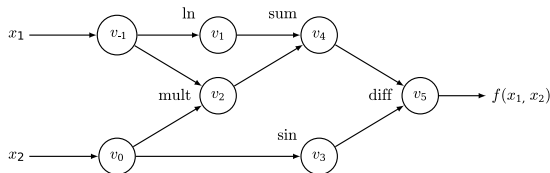
$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

Derivación automática: grafo de primitivas

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$



Derivación automática: modo directo



$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

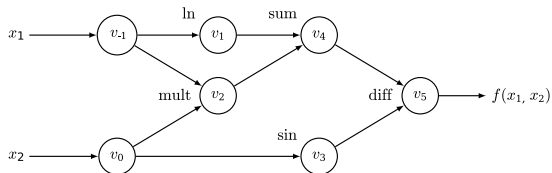
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo directo



$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

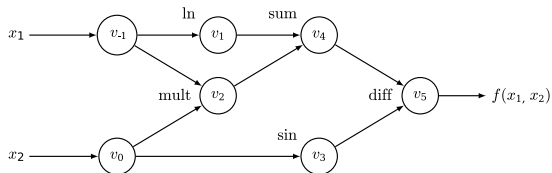
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo directo

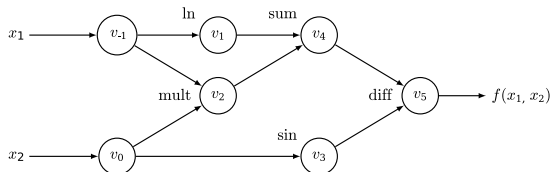


$$\dot{v}_i = \frac{\partial v_i}{\partial x_1}$$

$v_{-1} = x_1$	$= 2$
$v_0 = x_2$	$= 5$
$v_1 = \ln v_{-1}$	$= \ln 2$
$v_2 = v_{-1} \times v_0$	$= 2 \times 5$
$v_3 = \sin v_0$	$= \sin 5$
$v_4 = v_1 + v_2$	$= 0.693 + 10$
$v_5 = v_4 - v_3$	$= 10.693 + 0.959$
$y = v_5$	$= 11.652$

$\dot{v}_{-1} = \dot{x}_1$	$= 1$
$\dot{v}_0 = \dot{x}_2$	$= 0$
$\dot{v}_1 = \dot{v}_{-1}/v_{-1}$	$= 1/2$
$\dot{v}_2 = \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$
$\dot{v}_3 = \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$
$\dot{v}_4 = \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
$\dot{v}_5 = \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
$\dot{y} = \dot{v}_5$	$= 5.5$

Derivación automática: modo **reverso**



$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

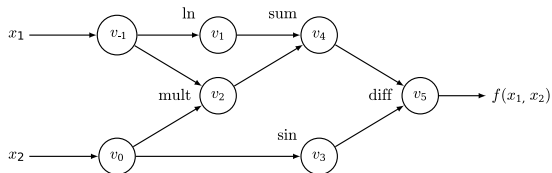
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

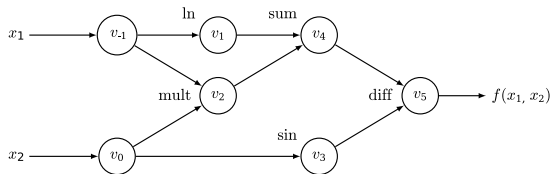
$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

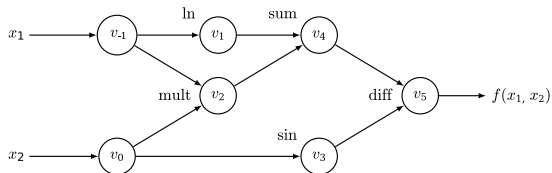
$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$\bar{v}_5 = \frac{\partial y}{\partial v_5} = \frac{\partial y}{\partial y}$$

$$y = v_5 = 11.652$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

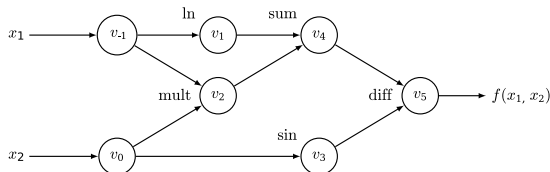
$$y = v_5 = 11.652$$

$$\bar{v}_4 = \frac{\partial y}{\partial v_4} = \frac{\partial y}{\partial v_5} \cdot \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \cdot 1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

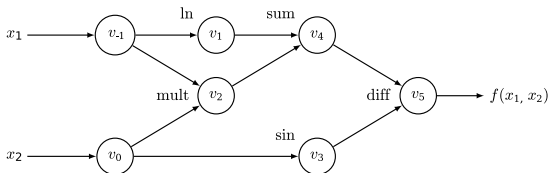
$$\bar{v}_3 = \frac{\partial y}{\partial v_3} = \frac{\partial y}{\partial v_5} \cdot \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \cdot (-1)$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{v}_1 = \frac{\partial y}{\partial v_1} = \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \cdot \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \cdot 1$$

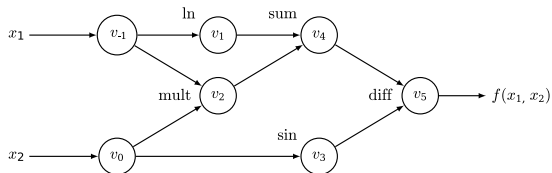
$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

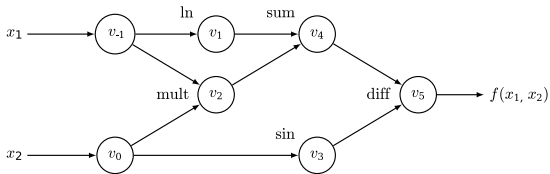
$$y = v_5 = 11.652$$

$$\bar{v}_2 = \frac{\partial y}{\partial v_2} = \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \cdot \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \cdot 1$$

$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2}$	$= \bar{v}_4 \times 1$	$= 1$
$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1}$	$= \bar{v}_4 \times 1$	$= 1$
$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3}$	$= \bar{v}_5 \times (-1)$	$= -1$
$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4}$	$= \bar{v}_5 \times 1$	$= 1$

$\bar{v}_5 = \bar{y}$	$= 1$
-----------------------	-------

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{v}_0^{(3)} = \frac{\partial y}{\partial v_0} = \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_3} \cdot \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \cdot \cos v_0$$

$$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$$

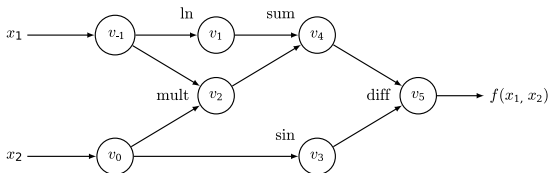
$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{v}_{-1}^{(2)} = \frac{\partial y}{\partial v_{-1}} = \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_2} \cdot \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \cdot v_0$$

$$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}}$$

$$= \bar{v}_2 \times v_0 = 5$$

$$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0}$$

$$= \bar{v}_3 \times \cos v_0 = -0.284$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2}$$

$$= \bar{v}_4 \times 1 = 1$$

$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1}$$

$$= \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3}$$

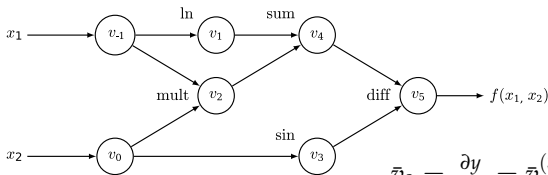
$$= \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4}$$

$$= \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

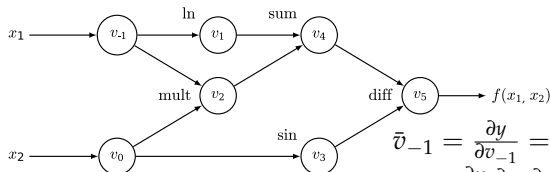
$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\begin{aligned} \bar{v}_0 &= \frac{\partial y}{\partial v_0} = \bar{v}_0^{(3)} + \bar{v}_0^{(2)} \\ &= \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_3} \frac{\partial v_3}{\partial v_0} + \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_2} \cdot \frac{\partial v_2}{\partial v_0} \\ &= \bar{v}_0^{(3)} + \bar{v}_2 \cdot v_{-1} \end{aligned}$$

$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0}$	$= \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$
$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}}$	$= \bar{v}_2 \times v_0 = 5$
$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0}$	$= \bar{v}_3 \times \cos v_0 = -0.284$
$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2}$	$= \bar{v}_4 \times 1 = 1$
$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1}$	$= \bar{v}_4 \times 1 = 1$
$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3}$	$= \bar{v}_5 \times (-1) = -1$
$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4}$	$= \bar{v}_5 \times 1 = 1$
$\bar{v}_5 = \bar{y}$	$= 1$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\begin{aligned} \bar{v}_{-1} &= \frac{\partial y}{\partial v_{-1}} = \bar{v}_{-1}^{(2)} + \bar{v}_{-1}^{(1)} \\ &= \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_2} \frac{\partial v_2}{\partial v_{-1}} + \frac{\partial y}{\partial v_5} \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_1} \cdot \frac{\partial v_1}{\partial v_{-1}} \\ &= \bar{v}_{-1}^{(2)} + \bar{v}_1 \cdot \frac{1}{v_{-1}} \end{aligned}$$

$$\bar{v}_{-1} = \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{v}_{-1} + \bar{v}_1 / v_{-1} = 5.5$$

$$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} = \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$$

$$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \times v_0 = 5$$

$$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$$

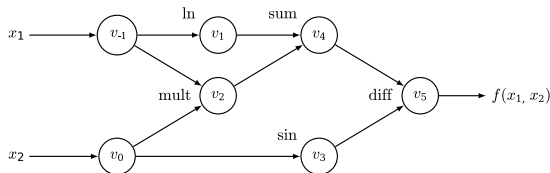
$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**



$$\bar{v}_i = \frac{\partial y}{\partial v_i}$$

$$v_{-1} = x_1 = 2$$

$$v_0 = x_2 = 5$$

$$v_1 = \ln v_{-1} = \ln 2$$

$$v_2 = v_{-1} \times v_0 = 2 \times 5$$

$$v_3 = \sin v_0 = \sin 5$$

$$v_4 = v_1 + v_2 = 0.693 + 10$$

$$v_5 = v_4 - v_3 = 10.693 + 0.959$$

$$y = v_5 = 11.652$$

$$\bar{x}_1 = \bar{v}_{-1}$$

$$= 5.5$$

$$\bar{x}_2 = \bar{v}_0$$

$$= 1.716$$

$$\bar{v}_{-1} = \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} = \bar{v}_{-1} + \bar{v}_1 / v_{-1} = 5.5$$

$$\bar{v}_0 = \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} = \bar{v}_0 + \bar{v}_2 \times v_{-1} = 1.716$$

$$\bar{v}_{-1} = \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} = \bar{v}_2 \times v_0 = 5$$

$$\bar{v}_0 = \bar{v}_3 \frac{\partial v_3}{\partial v_0} = \bar{v}_3 \times \cos v_0 = -0.284$$

$$\bar{v}_2 = \bar{v}_4 \frac{\partial v_4}{\partial v_2} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} = \bar{v}_4 \times 1 = 1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = \bar{v}_5 \times (-1) = -1$$

$$\bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = \bar{v}_5 \times 1 = 1$$

$$\bar{v}_5 = \bar{y} = 1$$

Derivación automática: modo **reverso**

Pasada hacia adelante  python

```

v_1 = x1 = 2          # 2
v0 = x2 = 5           # 5

v1 = math.log(v_1)    # 0.6931471805599453
v2 = v_1 * v0          # 10

v3 = math.sin(v0)      # -0.9589242746631385
v4 = v1 + v2           # 10.693147180559945

v5 = v4 - v3           # 11.652071455223084

y = v5                # 11.652071455223084

```

Derivación automática: modo **reverso**

Pasada hacia atrás 🐍 python

```

v5p = yp = 1                                # 1

v4p = v5p * 1                                # 1
v3p = v5p * -1                               # -1
v1p = v4p * 1                                # 1
v2p = v4p * 1                                # 1

v0p = v3p * math.cos(v0)                     # -0.28366218546322625
v_1p = v2p * v0                              # 5
v0p = v0p + v2p * v_1                         # 1.7163378145367738
v_1p = v_1p + v1p / v_1                       # 5.5

x2p = v0p                                    # 1.7163378145367738
x1p = v_1p                                   # 5.5

```

Derivación automática: modo **reverso**

Ahora guardando todo en la misma estructura

Pasada hacia adelante

```
x1 = [2., 0.]
```

```
x2 = [5., 0.]
```

```
v_1 = x1
```

```
v0 = x2
```

```
v1 = [math.log(v_1[0]), 0.]
```

```
v2 = [v_1[0] * v0[0], 0.]
```

```
v3 = [math.sin(v0[0]), 0.]
```

```
v4 = [v1[0] + v2[0], 0.]
```

```
v5 = [v4[0] - v3[0], 0.]
```

```
y = v5
```

Derivación automática: modo **reverso**

Ahora guardando todo en la misma estructura

Pasada hacia adelante

```
x1 = [2., 0.]
x2 = [5., 0.]
```

```
v_1 = x1
v0 = x2
```

```
v1 = [math.log(v_1[0]), 0.]
v2 = [v_1[0] * v0[0], 0.]
```

```
v3 = [math.sin(v0[0]), 0.]
v4 = [v1[0] + v2[0], 0.]
```

```
v5 = [v4[0] - v3[0], 0.]
```

```
y = v5
```

Pasada hacia atrás

```
y[1] = 1
v5[1] = y[1]
```

```
v4[1] = v5[1] * 1
v3[1] = v5[1] * -1
v1[1] = v4[1] * 1
v2[1] = v4[1] * 1
```

```
v0[1] = v3[1] * math.cos(v0[0])
v_1[1] = v2[1] * v0[0]
v0[1] = v0[1] + v2[1] * v_1[0]
v_1[1] = v_1[1] + v1[1] / v_1[0]
```

```
x2[1] = v0[1]
x1[1] = v_1[1]
```

Derivación automática: modo **reverso**



PyTorch

Derivación automática: modo **reverso**

Hacia adelante  python

```
x1 = 2.
```

```
x2 = 5.
```

```
v_1 = x1
```

```
v0 = x2
```

```
v1 = math.log(v_1)
```

```
v2 = v_1 * v0
```

```
v3 = math.sin(v0)
```

```
v4 = v1 + v2
```

```
v5 = v4 - v3
```

```
y = v5
```

Derivación automática: modo **reverso**

Hacia adelante  python Hacia adelante  PyTorch

```
x1 = 2.
```

```
x2 = 5.
```

```
v_1 = x1
```

```
v0 = x2
```

```
v1 = math.log(v_1)
```

```
v2 = v_1 * v0
```

```
v3 = math.sin(v0)
```

```
v4 = v1 + v2
```

```
v5 = v4 - v3
```

```
y = v5
```

```
x1 = torch.tensor(2., requires_grad=True)
```

```
x2 = torch.tensor(5., requires_grad=True)
```

```
v_1 = x1
```

```
v0 = x2
```

```
v1 = torch.log(v_1)
```

```
v2 = v_1 * v0
```

```
v3 = torch.sin(v0)
```

```
v4 = v1 + v2
```

```
v5 = v4 - v3
```

```
y = v5
```

Derivación automática: modo **reverso**

Hacia atrás  python

```
v5p = yp = 1
```

```
v4p = v5p * 1
```

```
v3p = v5p * -1
```

```
v1p = v4p * 1
```

```
v2p = v4p * 1
```

```
v0p = v3p * math.cos(v0)
```

```
v_1p = v2p * v0
```

```
v0p = v0p + v2p * v_1
```

```
v_1p = v_1p + v1p / v_1
```

```
x2p = v0p
```

```
x1p = v_1p
```

Derivación automática: modo **reverso**

Hacia atrás  python

```
v5p = yp = 1
```

```
v4p = v5p * 1
```

```
v3p = v5p * -1
```

```
v1p = v4p * 1
```

```
v2p = v4p * 1
```

```
v0p = v3p * math.cos(v0)
```

```
v_1p = v2p * v0
```

```
v0p = v0p + v2p * v_1
```

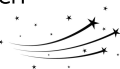
```
v_1p = v_1p + v1p / v_1
```

```
x2p = v0p
```

```
x1p = v_1p
```

Hacia atrás  PyTorch

```
y.backward()
```



Derivación automática: modo **reverso**

Código completo en  PyTorch

```
x1 = torch.tensor(2., requires_grad=True)
x2 = torch.tensor(5., requires_grad=True)
```

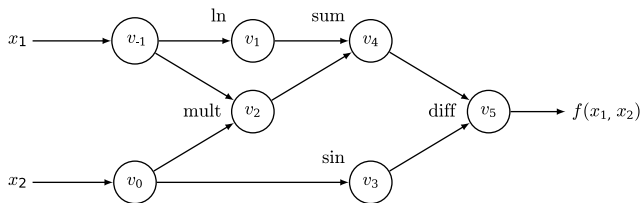
```
v_1 = x1
v0 = x2
v1 = torch.log(v_1)
v2 = v_1 * v0
v3 = torch.sin(v0)
v4 = v1 + v2
v5 = v4 - v3
```

```
y = v5
```

```
y.backward()
```

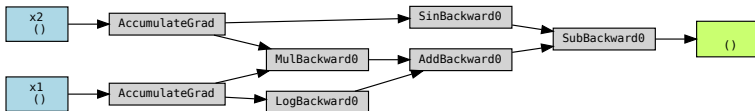
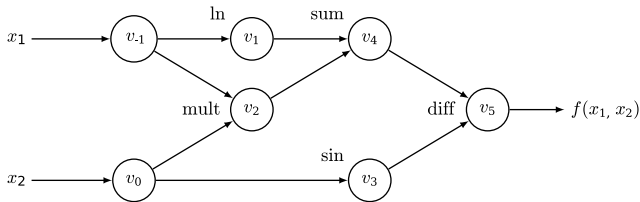
Derivación automática: modo **reverso**

Cómo se ve el grafo que arma  PyTorch?



Derivación automática: modo **reverso**

Cómo se ve el grafo que arma  PyTorch?



Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

```
v_1 = x1           ~ # tensor(2., requires_grad=True)
v0 = x2           ~ # tensor(5., requires_grad=True)
v1 = torch.log(v_1)
v2 = v_1 * v0
v3 = torch.sin(v0)
v4 = v1 + v2
v5 = v4 - v3

y = v5

y.backward()
```

Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

```
v_1 = x1                ~ # tensor(2., requires_grad=True)
v0 = x2                  ~ # tensor(5., requires_grad=True)
v1 = torch.log(v_1)      ~ # tensor(0.6931, grad_fn=<LogBackward0>)
v2 = v_1 * v0
v3 = torch.sin(v0)
v4 = v1 + v2
v5 = v4 - v3

y = v5

y.backward()
```

Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

```

v_1 = x1                ~> # tensor(2., requires_grad=True)
v0 = x2                  ~> # tensor(5., requires_grad=True)
v1 = torch.log(v_1)      ~> # tensor(0.6931, grad_fn=<LogBackward0>)
v2 = v_1 * v0             ~> # tensor(10., grad_fn=<MulBackward0>)
v3 = torch.sin(v0)
v4 = v1 + v2
v5 = v4 - v3

y = v5

y.backward()
```

Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

```
v_1 = x1           ~> # tensor(2., requires_grad=True)
v0 = x2           ~> # tensor(5., requires_grad=True)
v1 = torch.log(v_1) ~> # tensor(0.6931, grad_fn=<LogBackward0>)
v2 = v_1 * v0      ~> # tensor(10., grad_fn=<MulBackward0>)
v3 = torch.sin(v0) ~> # tensor(-0.9589, grad_fn=<SinBackward0>)
v4 = v1 + v2
v5 = v4 - v3

y = v5

y.backward()
```

Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

```

v_1 = x1           ~ # tensor(2., requires_grad=True)
v0 = x2            ~ # tensor(5., requires_grad=True)
v1 = torch.log(v_1) ~ # tensor(0.6931, grad_fn=<LogBackward0>)
v2 = v_1 * v0       ~ # tensor(10., grad_fn=<MulBackward0>)
v3 = torch.sin(v0)  ~ # tensor(-0.9589, grad_fn=<SinBackward0>)
v4 = v1 + v2        ~ # tensor(10.6931, grad_fn=<AddBackward0>)
v5 = v4 - v3        ~ # tensor(11.6521, grad_fn=<SubBackward0>)

y = v5             ~ # tensor(11.6521, grad_fn=<SubBackward0>)

y.backward()
```

Derivación automática: modo **reverso**

Verifiquemos qué se va guardando 🔥 PyTorch en cada paso

```

v_1 = x1           ~ # tensor(2., requires_grad=True)
v0 = x2            ~ # tensor(5., requires_grad=True)
v1 = torch.log(v_1) ~ # tensor(0.6931, grad_fn=<LogBackward0>)
v2 = v_1 * v0       ~ # tensor(10., grad_fn=<MulBackward0>)
v3 = torch.sin(v0)  ~ # tensor(-0.9589, grad_fn=<SinBackward0>)
v4 = v1 + v2        ~ # tensor(10.6931, grad_fn=<AddBackward0>)
v5 = v4 - v3        ~ # tensor(11.6521, grad_fn=<SubBackward0>)

y = v5             ~ # tensor(11.6521, grad_fn=<SubBackward0>)

y.backward()       ~ # tensor(5.5000) tensor(1.7163)

```

Bien! Magia ya tenemos...

Programación diferenciable: ∂P



Bien! Magia ya tenemos...

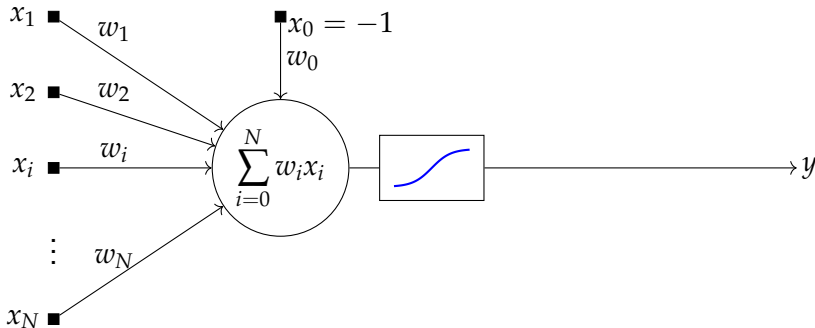
Programación diferenciable: ∂P



Pero cómo la usamos?

Cómo es que esta *magia* dio lugar
a la revolución de las redes neuronales profundas?

Perceptrón simple con activación sigmoidea



Sigamos el método tradicional

Queremos entrenar por gradiente, es decir

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_{\mathbf{w}} \xi(n),$$

y definimos como criterio el error cuadrático instantáneo

$$\xi(n) = (y_d(n) - y(n))^2 = (y_d(n) - \phi(\langle \mathbf{w}, \mathbf{x} \rangle))^2,$$

donde el producto interno $\langle \cdot, \cdot \rangle$ define la salida lineal $v = \langle \mathbf{w}, \mathbf{x} \rangle$, $\phi(\cdot)$ es la función de activación (no lineal) y $y_d(n)$ es la salida deseada (o referencia).

Sigamos el método tradicional

Para $\xi(n) = (y_d(n) - y(n))^2 = (y_d(n) - \phi(v))^2$,

aplicando la regla de la cadena obtenemos

$$\frac{\partial \xi(n)}{\partial w_i(n)} = \frac{\partial \xi(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)}$$

Sigamos el método tradicional

Para $\xi(n) = (y_d(n) - y(n))^2 = (y_d(n) - \phi(v))^2$,

aplicando la regla de la cadena obtenemos

$$\begin{aligned}
 \frac{\partial \xi(n)}{\partial w_i(n)} &= \frac{\partial \xi(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)} \\
 &= 2(y_d(n) - y(n))(-1) \cdot \phi'(v) \cdot x_i \\
 &= -2(y_d(n) - y(n)) \cdot \phi'(v) \cdot x_i.
 \end{aligned}$$

Sigamos el método tradicional

Para $\xi(n) = (y_d(n) - y(n))^2 = (y_d(n) - \phi(v))^2$,

aplicando la regla de la cadena obtenemos

$$\begin{aligned} \frac{\partial \xi(n)}{\partial w_i(n)} &= \frac{\partial \xi(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)} \\ &= 2(y_d(n) - y(n))(-1) \cdot \phi'(v) \cdot x_i \\ &= -2(y_d(n) - y(n)) \cdot \phi'(v) \cdot x_i. \end{aligned}$$

Y nos queda derivar la función de activación, que era la sigmoidea

$$\phi(v) = \frac{1}{1 + e^{-v}}$$

Sigamos el método tradicional

Una forma de encontrar la derivada es reescribiendo

$$\phi(v) = (1 + e^{-v})^{-1},$$

y aplicando la regla de la cadena

$$\phi'(v) = -(1 + e^{-v})^{-2} \cdot \frac{d}{dv}(1 + e^{-v}),$$

Sigamos el método tradicional

Una forma de encontrar la derivada es reescribiendo

$$\phi(v) = (1 + e^{-v})^{-1},$$

y aplicando la regla de la cadena

$$\phi'(v) = -(1 + e^{-v})^{-2} \cdot \frac{d}{dv}(1 + e^{-v}),$$

así que nos queda

$$\phi'(v) = -(1 + e^{-v})^{-2} \cdot (-e^{-v})$$

Sigamos el método tradicional

Una forma de encontrar la derivada es reescribiendo

$$\phi(v) = (1 + e^{-v})^{-1},$$

y aplicando la regla de la cadena

$$\phi'(v) = -(1 + e^{-v})^{-2} \cdot \frac{d}{dv}(1 + e^{-v}),$$

así que nos queda

$$\phi'(v) = -(1 + e^{-v})^{-2} \cdot (-e^{-v}) = \frac{e^{-v}}{(1 + e^{-v})^2}.$$

Sigamos el método tradicional

Además, para que quede en términos de la misma $\phi(v)$

$$\begin{aligned}
 \phi'(v) &= \frac{e^{-v}}{(1 + e^{-v})^2} \\
 &= \frac{1}{1 + e^{-v}} \cdot \frac{e^{-v}}{1 + e^{-v}} \\
 &= \frac{1}{1 + e^{-v}} \cdot \frac{\overbrace{-1 + 1}^0 + e^{-v}}{1 + e^{-v}} \\
 &= \frac{1}{1 + e^{-v}} \left(\frac{-1}{1 + e^{-v}} + \frac{1 + e^{-v}}{1 + e^{-v}} \right) \\
 &= \frac{1}{1 + e^{-v}} \left(\frac{-1}{1 + e^{-v}} + 1 \right) \\
 &= \frac{1}{1 + e^{-v}} \left(1 - \frac{1}{1 + e^{-v}} \right) = \phi(v)(1 - \phi(v))
 \end{aligned}$$

Sigamos el método tradicional

Y así obtenemos

$$\nabla_w \tilde{\zeta}(n) = -2 \cdot (y_d(n) - y(n)) \cdot \phi(v)(1 - \phi(v)) \cdot \mathbf{x},$$

y finalmente la actualización de los pesos queda

$$\begin{aligned} \mathbf{w}(n+1) &= \mathbf{w}(n) + \\ &\quad 2\mu \cdot (y_d(n) - y(n)) \cdot \\ &\quad y(n)(1 - y(n)) \cdot \mathbf{x} \end{aligned}$$

$$\partial P \equiv \text{[diagrama de flujo con flechas]}$$

 python

```

w = numpy.array(
    [0.0, 0.5, 0.5]
)

for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1 / (1 + numpy.exp(-v))

        e2 = (yd[n] - y) ** 2

        w = w + lr * 2 *
            (yd[n] - y) *
            y * (1 - y) * x[n]

```

$$\partial P \equiv \text{[diagrama de flujo con estrellas]}$$

 python

```
w = numpy.array(
    [0.0, 0.5, 0.5]
)
for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1 / (1 + numpy.exp(-v))

        e2 = (yd[n] - y) ** 2

        w = w + lr * 2 *
            (yd[n] - y) *
            y * (1 - y) * x[n]
```

 PyTorch

```
w = torch.tensor([0.0, 0.5, 0.5],
    dtype=torch.float64,
    requires_grad=True)
for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1 / (1 + torch.exp(-v))

        e2 = (yd[n] - y) ** 2
        e2.backward()

        with torch.no_grad():
            w_ = w - lr * w.grad
            w.copy_(w_)

        w.grad.zero_()
```

$$\partial P \equiv \text{[diagrama de flujo de gradiente]}$$

PyTorch

```
w = torch.tensor(...)

for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1/(1+torch.exp(-v))

        e2 = (yd[n]-y)**2
        e2.backward()

    with torch.no_grad():
        w_ = w - lr * w.grad
        w.copy_(w_)

w.grad.zero_()
```

$$\partial P \equiv \text{[diagrama de flujo con estrellas]}$$

PyTorch

```
w = torch.tensor(...)

for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1/(1+torch.exp(-v))

        e2 = (yd[n]-y)**2
        e2.backward()

    with torch.no_grad():
        w_ = w - lr * w.grad
        w.copy_(w_)

w.grad.zero_()
```

PyTorch más simple

```
w = torch.tensor(...)
optimizer=torch.optim.SGD([w],lr=0.1)

for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1/(1+torch.exp(-v))

        e2 = (yd[n]-y)**2
        e2.backward()

    optimizer.step()

optimizer.zero_grad()
```

$$\partial P \equiv \text{[diagrama de flujo con estrellas]}$$

 python

```
w = numpy.array(...)

for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1/(1+numpy.exp(-v))

        e2 = (yd[n]-y)**2

        w = w + lr * 2 *
            (yd[n]-y) *
            y*(1-y) * x[n]
```

 PyTorch simple

```
w = torch.tensor(...)
optimizer=torch.optim.SGD([w],lr=0.1)

for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = 1/(1+torch.exp(-v))

        e2 = (yd[n]-y)**2
        e2.backward()

        optimizer.step()
        optimizer.zero_grad()
```

Y si necesitamos otra medida de error?

Y si necesitamos otra medida de error?

Seguiremos entrenando por gradiente,

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \tilde{\zeta}(n),$$

pero ahora queremos usar otro criterio de error

$$\tilde{\zeta}(n) = - [y_d(n) \log y(n) + (1 - y_d(n)) \log(1 - y(n))],$$

Y si necesitamos otra medida de error?

Seguiremos entrenando por gradiente,

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \zeta(n),$$

pero ahora queremos usar otro criterio de error

$$\zeta(n) = -[y_d(n) \log y(n) + (1 - y_d(n)) \log(1 - y(n))],$$

Si fuera la misma función de activación, ya tenemos

$$\frac{\partial \zeta(n)}{\partial w_i(n)} = \frac{\partial \zeta(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)}$$

Y si necesitamos otra medida de error?

Seguiremos entrenando por gradiente,

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_{\mathbf{w}} \tilde{\zeta}(n),$$

pero ahora queremos usar otro criterio de error

$$\tilde{\zeta}(n) = - [y_d(n) \log y(n) + (1 - y_d(n)) \log(1 - y(n))],$$

Si fuera la misma función de activación, ya tenemos

$$\begin{aligned} \frac{\partial \tilde{\zeta}(n)}{\partial w_i(n)} &= \frac{\partial \tilde{\zeta}(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)} \\ &= \frac{\partial \tilde{\zeta}(n)}{\partial y(n)} \cdot \phi(v)(1 - \phi(v)) \cdot x_i \end{aligned}$$

Y si necesitamos otra medida de error?

Para $\xi(n) = -[y_d(n) \log y(n) + (1 - y_d(n)) \log(1 - y(n))]$,

tenemos

$$\begin{aligned}\frac{\partial \xi(n)}{\partial y(n)} &= - \left[\frac{y_d(n)}{y(n)} + \frac{1 - y_d(n)}{1 - y(n)} \cdot (-1) \right] = -\frac{y_d(n)}{y(n)} + \frac{1 - y_d(n)}{1 - y(n)} \\ &= \frac{-y_d(n)(1 - y(n)) + (1 - y_d(n))y(n)}{y(n)(1 - y(n))} \\ &= \frac{-y_d(n) + \cancel{y_d(n)y(n)} + y(n) - \cancel{y_d(n)y(n)}}{y(n)(1 - y(n))} \\ &= \frac{y(n) - y_d(n)}{y(n)(1 - y(n))}\end{aligned}$$

Y si necesitamos otra medida de error?

Así, la actualización de los pesos queda

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \cdot \frac{y(n) - y_d(n)}{y(n)(1 - y(n))} \cdot \mathbf{x}$$

$$\partial P \equiv \text{[diagrama de flujo con flechas]}$$



```
w = numpy.array(...
```

```
for ep in range(Nep):
```

```
    for n in range(N):
```

```
        v = w @ x[n]
```

```
        y = 1/(1+numpy.exp(-v))
```

```
    ce = -(
```

```
        yd[n]*numpy.log(y) +
```

```
        (1-yd[n])*numpy.log(1-y)
```

```
)
```

```
w = w - lr *
```

```
    (y-yd[n])/(y*(1-y)) *
```

```
    y*(1-y) * x[n]
```

$$\partial P \equiv \text{[diagrama de flujo con estrellas]}$$

 python

```
w = numpy.array(...)

for ep in range(Nep):
    for n in range(N):
        v = w @ x[n]
        y = 1/(1+numpy.exp(-v))

    ce = -(
        yd[n]*numpy.log(y) +
        (1-yd[n])*numpy.log(1-y)
    )

    w = w - lr *
        (y-yd[n])/(y*(1-y)) *
        y*(1-y) * x[n]
```

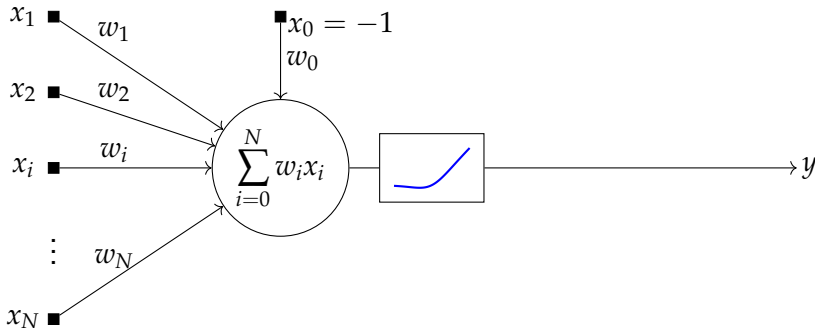
 PyTorch

```
w = torch.tensor(...)
optim = torch.optim.SGD([w],lr=0.1)
for ep in range(Nep):
    for n in range(N):
        v = w @ x[n]
        y = 1/(1+torch.exp(-v))

    ce = -(
        yd[n]*torch.log(y) +
        (1-yd[n])*torch.log(1-y)
    )
    ce.backward()

    optim.step()
    optim.zero_grad()
```

Y si queremos cambiar la función de activación?



Y si queremos cambiar la función de activación?

Para la función de activación

$$\phi(v) = \frac{v}{1 + e^{-v}},$$

si seguimos usando la regla de la cadena

$$\frac{\partial \xi(n)}{\partial w_i(n)} = \frac{\partial \xi(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)}$$

Y si queremos cambiar la función de activación?

Para la función de activación

$$\phi(v) = \frac{v}{1 + e^{-v}},$$

si seguimos usando la regla de la cadena

$$\begin{aligned} \frac{\partial \xi(n)}{\partial w_i(n)} &= \frac{\partial \xi(n)}{\partial y(n)} \cdot \frac{\partial y(n)}{\partial v(n)} \cdot \frac{\partial v(n)}{\partial w_i(n)} \\ &= -2(y_d(n) - y(n)) \cdot \phi'(v) \cdot x_i \end{aligned}$$

$$\phi'(v) = \dots = \frac{1 + e^{-v} + v(1 + e^{-v})}{(1 + e^{-v})^2} = \dots = y(n) (1 + v(n)(1 - y(n)))$$

$$\partial P \equiv \text{gradient vectors}$$

Pero ahora que ya sabemos usar la magia... directo a 🔥 PyTorch!

```
w = torch.tensor(...)
optimizer = torch.optim.SGD([w], lr=0.1)
```

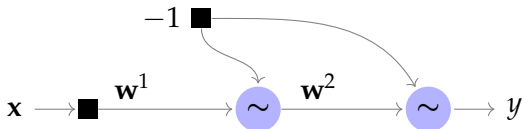
```
for ep in range(Nep):
    for n in range(N):

        v = w @ x[n]
        y = v / (1 + torch.exp(-v))

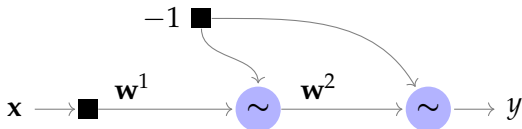
        ce = (y_d[n] - y)**2
        ce.backward()

    optimizer.step()
    optimizer.zero_grad()
```

Ahora queremos conectar más de una neurona



Ahora queremos conectar más de una neurona



$$\Delta w_{ji}^{(1)}(n) = \mu \delta_j^{(1)}(n) x_i(n)$$

$$\delta_j^{(1)}(n) = -\frac{\partial \xi(n)}{\partial y_j^{(1)}(n)} \frac{1}{2} y_j^{(1)}(n) (1 - y_j^{(1)}(n))$$

$$\delta_j^{(1)}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_k^2(n) \right\}}{\partial y_j^{(1)}(n)} \frac{1}{2} y_j^{(1)}(n) (1 - y_j^{(1)}(n))$$

$$\delta_j^{(1)}(n) = -\frac{1}{2} \sum_k \frac{\partial e_k^2(n)}{\partial y_j^{(1)}(n)} \frac{1}{2} y_j^{(1)}(n) (1 - y_j^{(1)}(n))$$

$$\delta_j^{(1)}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j^{(1)}(n)} \frac{1}{2} y_j^{(1)}(n) (1 - y_j^{(1)}(n))$$

$$\delta_j^{(1)}(n) = -\sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_k^{(2)}(n)} \frac{\partial y_k^{(2)}(n)}{\partial v_k^{(2)}(n)} \frac{\partial v_k^{(2)}(n)}{\partial y_j^{(1)}(n)} \frac{1}{2} y_j^{(1)}(n) (1 - y_j^{(1)}(n))$$

...

...

(1)

(2)

(2)

(2)

(1)

(1)

$$\partial P \equiv \text{[diagrama de flujo con estrellas]}$$

PyTorch

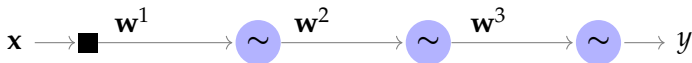
```
w1 = torch.tensor([[0.0, 0.5, 0.5]], ...
w2 = torch.tensor([0.0, 0.5], ...
m1b = torch.tensor([-1.0], ... # no requiere grad
optimizer = torch.optim.SGD([w1, w2], lr=0.1)
```

```
for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = 1/(1+torch.exp(-v1))
        y1 = torch.cat([m1b, y1a])
        v = w2 @ y1
        y = 1/(1+torch.exp(-v))

        ce = (y_d[n]-y)**2
        ce.backward()

    optimizer.step()
    optimizer.zero_grad()
```

Más capas neuronales?



$$\partial P \equiv \text{[diagrama de flujo con flechas curvas]}$$

PyTorch

```
w1 = torch.tensor([[0.0, 0.5, 0.5]], ...  
w2 = torch.tensor([[0.0, 0.5]], ...  
w3 = torch.tensor([[0.0, 0.5]], ...  
m1b = torch.tensor([-1.0], ...
```

```
optimizer = torch.optim.SGD([w1, w2, w3], lr=0.1)
```

$$\partial P \equiv \text{⤵}$$

PyTorch

```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = 1/(1+torch.exp(-v1))
        y1 = torch.cat([m1b, y1a])

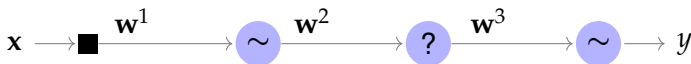
        v2 = w2 @ y1
        y2a = 1/(1+torch.exp(-v1))
        y2 = torch.cat([m1b, y2a])

        v = w3 @ y2
        y = 1/(1+torch.exp(-v))

        ce = (y_d[n]-y)**2
        ce.backward()
        optimizer.step()
        optimizer.zero_grad()

```

Y con otra función de activación en el medio?



$$\uparrow$$

$$\phi(v) = \frac{2}{1 + e^{-2v}} - 1$$

$$\partial P \equiv \text{[diagrama de flujo con estrellas y flechas]}$$

PyTorch

```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = 1/(1+torch.exp(-v1))
        y1 = torch.cat([m1b, y1a])

        v2 = w2 @ y1
        y2a = 2/(1+torch.exp(-2*v2))-1
        y2 = torch.cat([m1b, y2a])

        v = w3 @ y2
        y = 1/(1+torch.exp(-v))

        ce = (y_d[n]-y)**2
        ce.backward()
        optimizer.step()
        optimizer.zero_grad()

```

$$\partial P \equiv \text{[diagrama de flujo con flechas]}$$

PyTorch más simple

```
w1 = torch.tensor([[0.0, 0.5, 0.5]], ...
w2 = torch.tensor([[0.0, 0.5]], ...
w3 = torch.tensor([[0.0, 0.5]], ...
m1b = torch.tensor([-1.0], ...
```

```
optimizer = torch.optim.SGD([w1, w2, w3], lr=0.1)
```

```
sigmoid = torch.nn.Sigmoid()
```

```
tanh = torch.nn.Tanh()
```

```
mse = torch.nn.MSELoss()
```

$$\partial P \equiv \text{[diagrama de flujo con estrellas]}$$

PyTorch más simple

```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = sigmoid(v1)
        y1 = torch.cat([m1b, y1a])

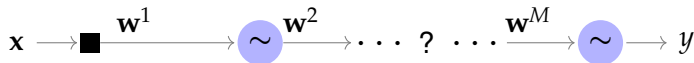
        v2 = w2 @ y1
        y2a = tanh(v2)
        y2 = torch.cat([m1b, y2a])

        v = w3 @ y2
        y = sigmoid(v1)

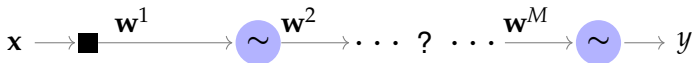
    ce = mse(y, y_d[n])
    ce.backward()
    optimizer.step()
    optimizer.zero_grad()

```

Y si necesitamos muchas capas más?



Y si necesitamos muchas capas más?



```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = sigmoid(v1)
        y1 = torch.cat([m1b, y1a])
        yh = y1
        for h in range(M-2):
            vh = wh[h] @ yh
            yha = tanh(vh)
            yh = torch.cat([m1b, yha])
        v = w3 @ yh
        y = sigmoid(v)

    ce = criterion(y, y_d[n])
    ce.backward()
    optimizer.step()
    optimizer.zero_grad()

```

Y una función de activación como esta?

$$\phi(v) = \begin{cases} v & \text{si } v > 0, \\ 0 & \text{si } v \leq 0. \end{cases}$$

Y una función de activación como esta?

$$\phi(v) = \begin{cases} v & \text{si } v > 0, \\ 0 & \text{si } v \leq 0. \end{cases}$$

```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = sigmoid(v1)
        y1 = torch.cat([m1b, y1a])
        yh = y1
        for h in range(M-2):
            vh = wh[h] @ yh
            if vh > 0:
                yha = vh
            else:
                yha = torch.zeros(1)
            yh = torch.cat([m1b, yha])
        v = w3 @ yh
        y = sigmoid(v)

    ce = criterion(y, y_d[n])
    ce.backward()...
```

Y una función de activación como esta?

$$\phi(v) = \begin{cases} v & \text{si } v > 0, \\ v/4 & \text{si } v \leq 0. \end{cases}$$

Y una función de activación como esta?

$$\phi(v) = \begin{cases} v & \text{si } v > 0, \\ v/4 & \text{si } v \leq 0. \end{cases}$$

```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = sigmoid(v1)
        y1 = torch.cat([m1b, y1a])
        yh = y1
        for h in range(M-2):
            vh = wh[h] @ yh
            if vh > 0:
                yha = vh
            else:
                yha = vh/4
            yh = torch.cat([m1b, yha])
        v = w3 @ yh
        y = sigmoid(v)

    ce = criterion(y, y_d[n])
    ce.backward()...
```

Y una función de activación como esta?

$$\phi(v) = \begin{cases} \sigma(v) & \text{si } v > 0, \\ v/4 & \text{si } v \leq 0. \end{cases}$$

Y una función de activación como esta?

$$\phi(v) = \begin{cases} \sigma(v) & \text{si } v > 0, \\ v/4 & \text{si } v \leq 0. \end{cases}$$

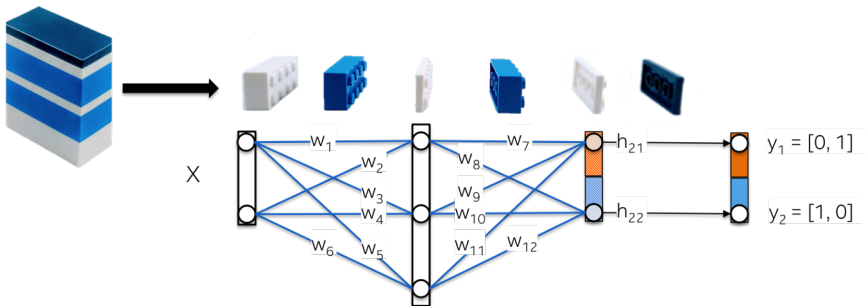
```

for ep in range(Nep):
    for n in range(N):
        v1 = w1 @ x[n]
        y1a = sigmoid(v1)
        y1 = torch.cat([m1b, y1a])
        yh = y1
        for h in range(M-2):
            vh = wh[h] @ yh
            if vh > 0:
                yha = sigmoid(vh)
            else:
                yha = vh/4
            yh = torch.cat([m1b, yha])
        v = w3 @ yh
        y = sigmoid(v)

ce = criterion(y, y_d[n])
ce.backward()...
```

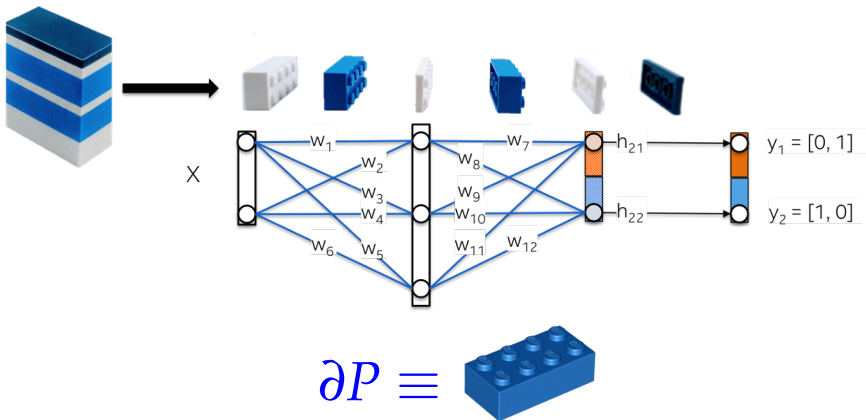
Programación diferenciable:

el Lego de las arquitecturas neuronales profundas

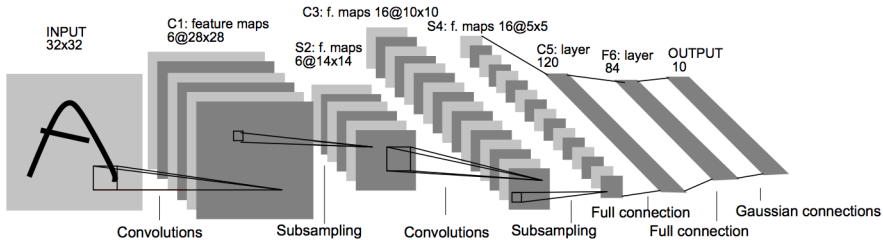


Programación diferenciable:

el Lego de las arquitecturas neuronales profundas

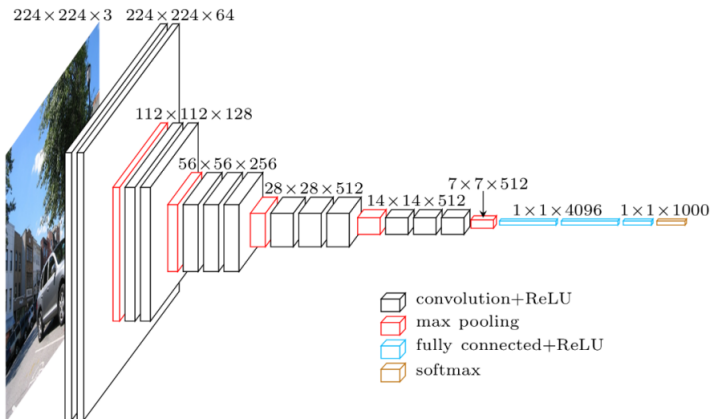


LeNet-5





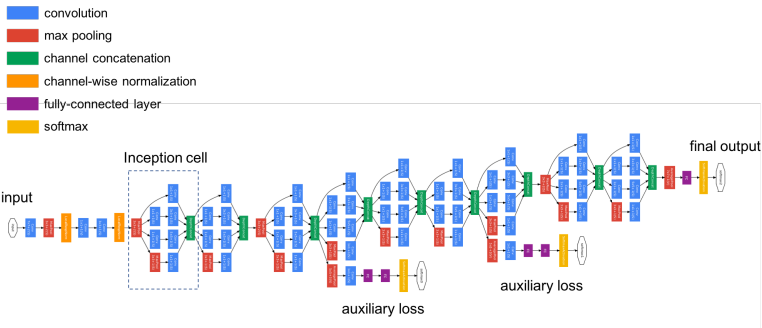
VGG-16





$$\partial P \equiv \text{blue brick icon}$$

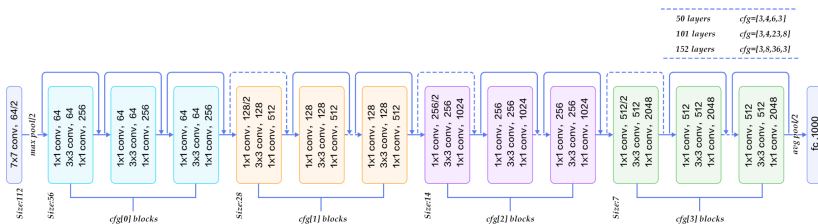
Inception



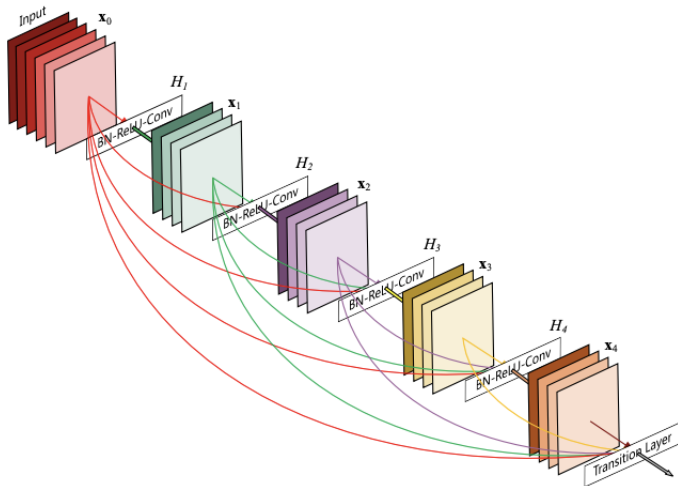


$$\partial P \equiv \text{LEGO}$$

ResNet



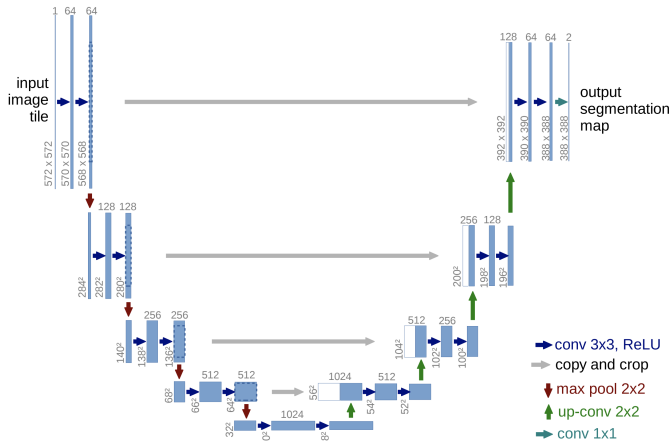
DenseNet





$$\partial P \equiv \text{blue brick icon}$$

U-Net



Diseños automáticos

